# When Keys Collide -- Chaining, Probing, and Resolution

## Lecture 17

Data Structures and Algorithms

## Where We Left Off

In the previous lecture, we discovered **Hash Functions** -- the magical mapping from arbitrary keys to array indices.

We learned that:

1. Hash functions provide O(1) expected time lookup by directly calculating an index.
2. We can use methods like the **Division Method** or the **Multiplication Method**.
3. **Collisions** (two distinct keys producing the same hash value) are mathematically inevitable due to the Pigeonhole Principle.

Today, we confront those collisions head-on.

**How do we build a data structure that survives when multiple keys demand the exact same slot?**

# Part 1: The Hash Table Data Structure

## The Hash Table Concept

A **Hash Table** consists of an array of $m$ **slots** (or **buckets**). Each slot stores a key-value pair (or a pointer to one).

The core operations we want to support in O(1) average time:

- `insert(key, value)`
- `search(key)`
- `delete(key)`

For every operation, the hash function $h(key)$ determines which slot we map to.

### The Dream vs The Reality

- **The Dream:** Every key maps to a unique slot. All operations are strictly O(1) and life is simple.
- **The Reality:** Collisions happen. Sometimes `h(k1) == h(k2)`. We need a strategy.

## The Two Philosophies

When a collision occurs, what should we do with the "new" key that wants to occupy an already-taken slot?

There are two major philosophies to handle this:

1. **"Let everyone share the same address"**
   Instead of storing just one item per slot, let slots hold multiple items. We can do this using Linked Lists.
   **This is called Separate Chaining.**

2. **"Find them a new address"**
   If a slot is full, we look for another empty slot inside the table itself using a fixed sequence of steps.
   **This is called Open Addressing.**

Let us explore both in detail.

# Part 2: Separate Chaining

## How Separate Chaining Works

In **Separate Chaining** (also called Open Hashing), each slot in the hash table array does not hold a value directly. Instead, it holds a pointer to a **linked list**.

When $k_1, k_2$, and $k_3$ all map to the same slot, we simply attach them to the linked list at that slot.

### Operations

- **Insert:** Compute $h(k)$, and insert the key at the beginning (or end) of the linked list at slot $h(k)$. **Time: O(1)**
- **Search:** Compute $h(k)$, go to slot $h(k)$, and traverse the linked list to find the key.
- **Delete:** Compute $h(k)$, search the list for the key, and remove the node from the linked list.

If the hash function distributes the items reasonably well, the lists stay very short.

## Separate Chaining: Worked Example

Assume a table size $m = 7$ and hash function $h(k) = k \mod 7$.

Let's insert the keys: 15, 11, 27, 8, 12, 22, 35, 4

| Key | $k \mod 7$ | Resulting Slot |
|---|---|---|
| 15 | 1 | Slot 1 |
| 11 | 4 | Slot 4 |
| 27 | 6 | Slot 6 |
| 8 | 1 | Slot 1 (Collision!) |
| 12 | 5 | Slot 5 |
| 22 | 1 | Slot 1 (Collision!) |
| 35 | 0 | Slot 0 |
| 4 | 4 | Slot 4 (Collision!) |

## Separate Chaining: Visualizing the Table

After all insertions, our hash table (an array of pointers) looks like this:

```
Slot 0: [ 35 ] --> NULL
Slot 1: [ 15 ] --> [ 8 ] --> [ 22 ] --> NULL
Slot 2: NULL
Slot 3: NULL
Slot 4: [ 11 ] --> [ 4 ] --> NULL
Slot 5: [ 12 ] --> NULL
Slot 6: [ 27 ] --> NULL
```

### Scenario: Search for 22

1. Compute $h(22) = 22 \mod 7 = 1$.
2. Go to Slot 1.
3. Traverse the list: compare with 15 (no), compare with 8 (no), compare with 22 (found!). Takes 3 comparisons.

*Notice that Slot 1 has grown into a small "chain" containing everything that hashed to 1.*

## Analysis of Separate Chaining

How efficient is this approach? It depends on how full the table is.

We define the **Load Factor ($\alpha$)**:

$$\alpha = \frac{n}{m} = \frac{\text{number of elements}}{\text{number of slots}}$$

- The average length of any chain is exactly $\alpha$.
- A successful search compares an average of $1 + \alpha/2$ nodes.
- An unsuccessful search compares an average of $\alpha$ nodes.

**The Beauty of Chaining:**

The hash table can support $\alpha > 1$. If you insert 20 items into 10 slots ($\alpha = 2$), every list has length 2 on average. Performance degrades "gracefully."

**Worst case:** All $n$ items hash to the exact same slot. Operations degrade to O(n) (equivalent to a regular linked list).

# Part 3: Open Addressing

## The Concept of Open Addressing

What if we don't want to deal with pointers and linked lists? Pointers use extra memory and jumping through linked lists can be slow due to CPU caching.

In **Open Addressing** (Closed Hashing):

- All elements are stored **directly inside the hash table array**.
- Each slot holds at most one element.
- If we try to insert an element and the slot is already occupied, we **probe** for an empty slot following a systematic check.

Since we never store more than one item per slot, the table can never hold more items than its size. In Open Addressing: $\alpha$ **must be** $< 1$.

## Strategy 1: Linear Probing

The simplest probing strategy. If the slot $h(k)$ is occupied, try the next one.
We probe systematically: try $h(k)$, then $h(k)+1$, then $h(k)+2$, wrapping around to the beginning if we reach the end of the array.

### Probe sequence formula:

$$h(k,i) = (h(k) + i) \mod m$$

*(where $i = 0, 1, 2, \ldots$ represents the attempt number)*

### Worked Example:

Table size $m = 10$, hash function $h(k) = k \mod 10$.
Insert: 48, 18, 28, 8, 38

Let's see what happens step by step.

## Linear Probing Example Trace

**Insert 48, 18, 28, 8, 38** into a table of 10 slots:

1. **Insert 48:** $h(48) = 8$. Slot 8 is empty. [Place 48 in slot 8]
2. **Insert 18:** $h(18) = 8$. Slot 8 occupied. Probe next $(8 + 1) = 9$. Empty. [Place 18 in slot 9]
3. **Insert 28:** $h(28) = 8$. Slot 8 occupied. Probe next $(8 + 1) = 9$. Occupied. Probe next $(9 + 1) \to 0$. Empty. [Place 28 in slot 0]
4. **Insert 8:** $h(8) = 8$. Slot 8, 9, 0 occupied. Find empty at slot 1. [Place 8 in slot 1]
5. **Insert 38:** $h(38) = 8$. Slot 8, 9, 0, 1 occupied. Find empty at slot 2. [Place 38 in slot 2]

```
Table:
[ 28 ][ 8 ][ 38 ][    ][    ][    ][    ][    ][ 48 ][ 18 ]
  0     1    2     3    4     5     6     7     8     9
```

## The Problem with Linear Probing

Linear probing is simple and very friendly to CPU caches. However, it suffers from a massive flaw: **Primary Clustering**.

Notice how long stretches of occupied slots (clusters) form.

- The cluster in our example stretches from slot 8 around to slot 2.
- If we now hash a brand-new key to slot 9, 0, 1, or 2, it will collide with the cluster and make it *even bigger*.

It is like a traffic jam. Once cars slow down and bunch together, any new car approaching from behind is forced to join the jam, making the jam longer.

**Primary Clustering causes the search and insertion time to degrade drastically as the load factor increases.**

## Strategy 2: Quadratic Probing

To stop long, contiguous clusters from forming, we can increase the gap between probes quadratically.

**Probe sequence formula:**

$$h(k, i) = (h(k) + c_1 \cdot i + c_2 \cdot i^2) \mod m$$

*Commonly simplified to:* $h(k, i) = (h(k) + i^2) \mod m$

**How it works:**

Instead of checking slots at distance 1, 2, 3, 4...
We check slots at distance $1^2 = 1, 2^2 = 4, 3^2 = 9, 4^2 = 16...$

By taking larger and larger jumps, a collision at slot $h(k)$ immediately causes the next item to skip over any existing linear cluster.

## Strategy 3: Double Hashing

Quadratic probing solves *primary* clustering, but it still has a subtle flaw: any two keys that have the same initial hash ($h(k_1) == h(k_2)$) will follow the **exact same** probe sequence. This is called *Secondary Clustering*.

We want the probe sequence itself to depend on the key.

**Double Hashing** uses two independent hash functions:

- $h_1(k)$ determines the initial starting position.
- $h_2(k)$ determines the **jump step size**.

**Probe sequence formula:**

$$h(k, i) = (h_1(k) + i \cdot h_2(k)) \mod m$$

If two keys collide initially, their second hash function $h_2(k)$ will likely be different, making them follow completely different paths! This efficiently scatters items throughout the table, eliminating both forms of clustering.

## The Deletion Problem in Open Addressing

Deleting an element in open addressing is tricky.

Suppose we used Linear Probing:

1. We inserted A at slot 5.
2. We inserted B. It hashed to 5, collided with A, and was placed at slot 6.
3. Now, we **delete A** by emptying slot 5.
4. Next, we **search for B**. The hash function tells us to start at slot 5.
5. The search sees slot 5 is empty, and instantly concludes: "B is not in the table!" **(This is WRONG!)**

### The Solution: Tombstones

When we delete an item from an open-addressed table, we do not leave it completely empty. We leave a special marker called a **Tombstone** (e.g., "Deleted").

During a search, if we hit a Tombstone, we know: "Something used to be here, keep probing!" New insertions can safely overwrite Tombstones.

# Part 4: Interactive Tools & Comparisons

## Interactive Visualization Tools

To truly see chaining and probing in action, we can use these fantastic online visualization tools from the University of San Francisco:

1. **Separate Chaining (Open Hashing):**
   https://www.cs.usfca.edu/~galles/visualization/OpenHash.html
2. **Open Addressing (Closed Hashing):**
   https://www.cs.usfca.edu/~galles/visualization/ClosedHash.html

*(I will load these up for us to try inserting elements and watch the clustering and resolving behaviors in real-time.)*

## Comparison Summary: Open Addressing Methods

| Feature | Linear Probing | Quadratic Probing | Double Hashing |
|---|---|---|---|
| Step Size | Fixed step of +1 | Increases as $i^2$ | Based on key $h_2(k)$ |
| Clustering | Primary Clustering | Secondary Clustering | No Clustering |
| Computation | Very fast | Moderate | Slower (2 hashes) |
| Cache Friendly? | Excellent | Good | Poor (Random jumps) |

In practice:
Linear Probing is often the fastest *if* the table remains mostly empty ($\alpha \leq 0.5$) because modern CPUs process adjacent array slots extremely quickly (cache locality).
For higher load factors, Double Hashing provides the best distribution.

# Key Takeaways

## Wrap-up

**1. The Hash Table Data Structure:**

Combines an array with a hash function. It achieves O(1) performance by computing the memory address directly, but must deal with collisions.

**2. Separate Chaining:**

Every slot points to a Linked List. Colliding elements get appended. Easy to implement, tolerates load factors $> 1$, memory overhead for pointers.

**3. Open Addressing:**

All elements stay inside the array. No pointers overhead. Limits load factor $< 1$.

- **Linear Probing:** Simple, probe next slot. Suffers Primary Clustering.
- **Quadratic Probing:** Jump by $i^2$. Solves primary clustering.
- **Double Hashing:** Jump by a second hash function. Solves all clustering.
- **Tombstones:** Required to make deletions work without breaking searches.

## Coming Up Next

We have seen what happens when collisions occur. But what happens when the hash table gets **full**?

Even Separate Chaining becomes slow if $\alpha$ reaches 10, because every list has 10 elements to search through. Open Addressing completely breaks when $\alpha$ reaches 1.

In the next lecture, we will study:

- **Load Factor Analysis:** Exactly how slow do operations get?
- **Rehashing:** How do we resize a hash table dynamically while keeping operations O(1) amortized?
- **Real-World Applications:** How Python dictionaries, Java HashMaps, Compilers, and Caching Systems use hash tables to power modern computing.